

RANGKUMAN BELAJAR REPLIKASI POSTGRESQL

Panduan Praktis Setup Physical & Logical Replication (Tabel hingga Kolom) menggunakan Docker

1. Ikhtisar Konsep: Physical vs Logical Replication

PostgreSQL menyediakan dua metode utama untuk melakukan replikasi data ke node sekunder (Slave/Replica). Pemilihan metode ini sangat menentukan bagaimana fleksibilitas struktur data di sisi replica nantinya.

Karakteristik	Physical (Streaming) Replication	Logical Replication
Mekanisme	Menyalin file biner secara mentah (<i>byte-by-byte</i>) berdasarkan log WAL.	Mengirim perubahan baris data logis (INSERT/UPDATE/DELETE) lewat skema Publisher-Subscriber.
Cakupan Data	Seluruh cluster database (100% sama persis, semua tabel dan skema).	Sangat selektif (Bisa memilih tabel tertentu atau kolom tertentu saja).
Sifat Replica	<i>Read-Only</i> penuh. Transaksi write ditolak langsung oleh engine database.	<i>Read-Only</i> untuk tabel replikasi, namun bisa menjalankan <i>Write</i> untuk tabel lokal baru.
Kustomisasi Index	Tidak Bisa. Index harus sama persis dengan Primary karena menyalin fisik disk.	Bisa. Bebas menambah index khusus di sisi Replica untuk kebutuhan optimasi query analitik.
Versi PostgreSQL	Wajib menggunakan versi mayor yang sama antara Primary dan Replica.	Bisa lintas versi mayor (Sangat berguna untuk skenario upgrade database).

Intisari Aturan Main: Jika tujuan utama adalah *High Availability / Disaster Recovery* (cadangan server mati), pilih **Physical**. Jika tujuan utama adalah memisahkan database laporan yang butuh optimasi query khusus atau filter data sensitif, pilih **Logical**.

2. Panduan Setup Physical (Streaming) Replication

Metode ini menduplikasi seluruh isi database utama menggunakan utility bawaan `pg_basebackup`.

Langkah 1: Membuat File `docker-compose.yml`

```
version: '3.8'
services:
  pg_primary:
    image: postgres:15-alpine
    container_name: pg_primary
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - pg_primary_data:/var/lib/postgresql/data
      - ./init-primary.sh:/docker-entrypoint-initdb.d/init-primary.sh

  pg_replica:
    image: postgres:15-alpine
    container_name: pg_replica
    environment:
      PGPASSWORD: mypassword
    ports:
      - "5433:5432"
    depends_on:
      - pg_primary
    entrypoint: >
      bash -c "
        until pg_isready -h pg_primary -p 5432 -U myuser; do echo 'Waiting for primary...';
        sleep 1; done;
        if [ ! -s /var/lib/postgresql/data/PG_VERSION ]; then
          pg_basebackup -h pg_primary -D /var/lib/postgresql/data -U replication_user -v -P -
X stream -Fp -R
        fi;
        exec docker-entrypoint.sh postgres
      "
    volumes:
      - pg_replica_data:/var/lib/postgresql/data

volumes:
  pg_primary_data:
  pg_replica_data:
```

Langkah 2: Membuat Script Inisialisasi `init-primary.sh`

Script ini berjalan otomatis di sisi Primary untuk membuat role khusus replikasi dan membuka hak akses jaringan:

```
#!/bin/bash
set -e
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" <<-EOSQL
    CREATE ROLE replication_user WITH REPLICATION LOGIN PASSWORD 'mypassword';
EOSQL
echo "host replication replication_user all md5" >> "$PGDATA/pg_hba.conf"
```

Pastikan menjalankan perintah `chmod +x init-primary.sh` sebelum menaikkan container.

3. Panduan Setup Logical Replication

Pada tingkat basis ini, kita akan menghubungkan dua container independen dan mengaktifkan parameter `wal_level = logical` di sisi Publisher.

Langkah 1: Membuat File `docker-compose.yml`

```
version: '3.8'
services:
  pg_publisher:
    image: postgres:15-alpine
    container_name: pg_publisher
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    command: postgres -c wal_level=logical

  pg_subscriber:
    image: postgres:15-alpine
    container_name: pg_subscriber
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydb
    ports:
      - "5433:5432"
    depends_on:
      - pg_publisher
```

Langkah 2: Eksekusi Perintah SQL Inisialisasi

1. **Buat tabel secara manual di kedua sisi** (Logical tidak menyalin skema secara otomatis):

```
-- Jalankan di Publisher (Port 5432) dan Subscriber (Port 5433)
CREATE TABLE produk (
    id SERIAL PRIMARY KEY,
    nama_produk VARCHAR(100),
    harga NUMERIC
);
```

2. Tambahkan Index Khusus di Subscriber saja (Opsional untuk Optimasi):

```
-- Jalankan HANYA di Subscriber (Port 5433)
CREATE INDEX idx_harga_produk ON produk(harga);
```

3. Aktifkan Publikasi di sisi Publisher:

```
-- Jalankan di Publisher (Port 5432)
CREATE PUBLICATION pub_produk FOR TABLE produk;
```

4. Aktifkan Berlangganan di sisi Subscriber:

```
-- Jalankan di Subscriber (Port 5433)
CREATE SUBSCRIPTION sub_produk
CONNECTION 'host=pg_publisher port=5432 user=myuser password=mypassword dbname=mydb'
PUBLICATION pub_produk;
```

4. Manajemen Tingkat Lanjut Logical Replication

A. Menambahkan Tabel Baru ke Skema Replikasi yang Sudah Ada

Jika ada tabel baru (misal: tabel `kategori`), alurnya adalah:

1. Buat tabel `kategori` dengan Primary Key di Subscriber (5433).
2. Buat tabel `kategori` dengan Primary Key di Publisher (5432).
3. Masukkan tabel baru ke publikasi lama (Jalankan di Publisher):

```
ALTER PUBLICATION pub_produk ADD TABLE kategori;
```

4. Segarkan konfigurasi agar subscriber mulai menarik data baru (Jalankan di Subscriber):

```
ALTER SUBSCRIPTION sub_produk REFRESH PUBLICATION;
```

B. Membuat Jalur Replikasi Baru (Opsi Membuat Baru vs ALTER)

- **Kapan pakai ALTER:** Bila tabel baru tersebut masih berada dalam satu kesatuan fungsi aplikasi yang sama (Contoh: tabel `kategori` melengkapi tabel `produk`).

- **Kapan buat baru (CREATE PUBLICATION baru):** Bila tabel baru memiliki konteks logika yang berbeda (Misal: modul `transaksi`), atau ingin direplikasi ke target database yang berbeda lagi. Mencegah satu error merusak aliran seluruh data.

5. Skenario Replikasi Spesifik Tingkat Kolom (Column Filter)

Fitur ini membatasi pemaparan data sensitif (seperti password atau token) agar tidak terkirim ke database replica.

Langkah Implementasi:

1. Definisikan tabel lengkap di sisi Publisher (5432):

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50),  
    email VARCHAR(100),  
    password_hash VARCHAR(255) -- Kolom Sensitif  
);
```

2. Definisikan tabel terbatas tanpa kolom sensitif di Subscriber (5433):

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50),  
    email VARCHAR(100)  
);
```

3. Buat Publikasi dengan menyertakan nama kolom di Publisher:

```
CREATE PUBLICATION pub_users_aman FOR TABLE users (id, username, email);
```

4. Buat Subscription baru di Subscriber untuk mengikat data tersebut:

```
CREATE SUBSCRIPTION sub_users_aman  
CONNECTION 'host=pg_publisher port=5432 user=myuser password=mypassword dbname=mydb'  
PUBLICATION pub_users_aman;
```

Aturan Kritis Penapisan Kolom: Kolom yang berstatus sebagai **Primary Key** wajib disertakan di dalam tanda kurung deklarasi publikasi. Jika dilewatkan, proses replikasi logis untuk aksi `UPDATE` dan `DELETE` akan otomatis mogok karena sistem kehilangan penanda identitas baris data.